

Image Classification with CNN and Transfer Learning

```
import os
import random
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

from PIL import Image, UnidentifiedImageError

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, Sequential
from tensorflow.keras.layers import (
    Conv2D, MaxPooling2D, Flatten, Dense,
    BatchNormalization, Dropout, Activation,
    GlobalAveragePooling2D
)
from tensorflow.keras.models import Model
from tensorflow.keras.applications import VGG16
from sklearn.metrics import classification_report

print("TensorFlow version:", tf.__version__)
```

TensorFlow version: 2.19.0

Dataset Path

```
import os
print(os.path.exists("/content/FruitinAmazon.zip"))
```

True

```
from google.colab import files
files.upload()
```

[Show hidden output](#)

```
import zipfile

zip_path = "/content/FruitinAmazon.zip"
extract_path = "/content/dataset"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset extracted!")
```

Dataset extracted!

```
import os
print(os.listdir("/content/dataset"))
```

['FruitinAmazon']

```
print(os.listdir("/content/dataset/FruitinAmazon"))
```

['test', 'train']

```
train_dir = "/content/dataset/train"
test_dir = "/content/dataset/test"
```

```
train_dir = "/content/dataset/FruitinAmazon/train"
test_dir = "/content/dataset/FruitinAmazon/test"
```

```
print(os.listdir(train_dir))
```

['tucuma', 'pupunha', 'guarana', 'graviola', 'cupuacu', 'acai']

Dataset Understanding and Visualization

```

class_names = sorted(
    [d for d in os.listdir(train_dir) if os.path.isdir(os.path.join(train_dir, d))]
)

if not class_names:
    print("No class directories found in the train folder!")
else:
    print(f"Found {len(class_names)} classes: {class_names}")

```

Found 6 classes: ['acai', 'cupuacu', 'graviola', 'guarana', 'pupunha', 'tucuma']

```

corrupted_images = []

for class_name in class_names:
    class_path = os.path.join(train_dir, class_name)

    if os.path.isdir(class_path):
        images = os.listdir(class_path)

        for img_name in images:
            img_path = os.path.join(class_path, img_name)

            try:
                with Image.open(img_path) as img:
                    img.verify()
            except (IOError, UnidentifiedImageError):
                corrupted_images.append(img_path)

if corrupted_images:
    print("\nCorrupted Images Found:")
    for img in corrupted_images:
        print(img)
else:
    print("\nNo corrupted images found.")

```

No corrupted images found.

```

class_counts = {}

for class_name in class_names:
    class_path = os.path.join(train_dir, class_name)

    if os.path.isdir(class_path):
        images = [
            img for img in os.listdir(class_path)
            if img.lower().endswith((".png", ".jpg", ".jpeg"))
        ]
        class_counts[class_name] = len(images)

print("\nClass Distribution:")
print("=" * 45)
print(f"{'Class Name':<25}{'Valid Image Count':>15}")
print("=" * 45)
for class_name, count in class_counts.items():
    print(f"{class_name:<25}{count:>15}")
print("=" * 45)

```

```

Class Distribution:
=====
Class Name          Valid Image Count
=====
acai                  15
cupuacu               15
graviola              15
guarana               15
pupunha               15
tucuma                15
=====

```

```

selected_images = []
selected_labels = []

```

```

for class_name in class_names:
    class_path = os.path.join(train_dir, class_name)

    if os.path.isdir(class_path):
        images = [
            img for img in os.listdir(class_path)
            if img.lower().endswith((".png", ".jpg", ".jpeg"))
        ]

        if images:
            selected_img = os.path.join(class_path, random.choice(images))
            selected_images.append(selected_img)
            selected_labels.append(class_name)

```

```

num_classes = len(selected_images)
cols = (num_classes + 1) // 2
rows = 2

fig, axes = plt.subplots(rows, cols, figsize=(12, 6))

for i, ax in enumerate(axes.flat):
    if i < num_classes:
        img = mpimg.imread(selected_images[i])
        ax.imshow(img)
        ax.set_title(selected_labels[i])
        ax.axis("off")
    else:
        ax.axis("off")

plt.tight_layout()
plt.show()

```

acai



cupuacu



graviola



guarana



pupunha



tucuma



Data Generation and Pre-processing

```

image_size = (28, 28)
batch_size = 32

train_ds = keras.utils.image_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset="training",
    seed=1337,

```

```
        image_size=image_size,
        batch_size=batch_size,
        color_mode="grayscale"
    )

    val_ds = keras.utils.image_dataset_from_directory(
        train_dir,
        validation_split=0.2,
        subset="validation",
        seed=1337,
        image_size=image_size,
        batch_size=batch_size,
        color_mode="grayscale"
    )
```

Found 90 files belonging to 6 classes.
Using 72 files for training.
Found 90 files belonging to 6 classes.
Using 18 files for validation.

Create test dataset

```
test_ds = keras.utils.image_dataset_from_directory(
    test_dir,
    image_size=image_size,
    batch_size=batch_size,
    shuffle=False,
    color_mode="grayscale"
)
```

Found 30 files belonging to 6 classes.

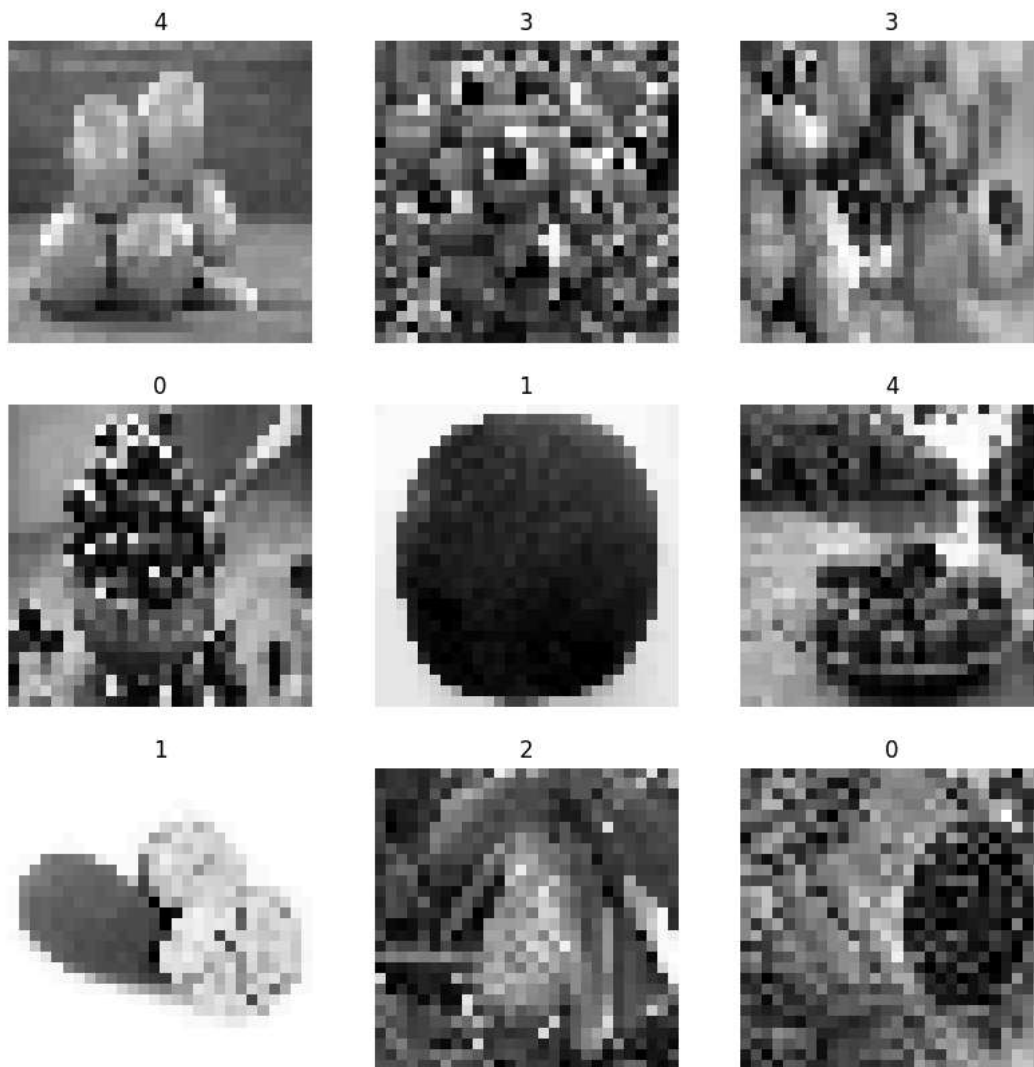
```
for images, labels in train_ds.take(1):
    print("Images shape:", images.shape)
    print("Labels shape:", labels.shape)
```

Images shape: (32, 28, 28, 1)
Labels shape: (32,)

```
plt.figure(figsize=(10, 10))

for images, labels in train_ds.take(1):
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(np.array(images[i]).astype("uint8").squeeze(), cmap="gray")
        plt.title(int(labels[i]))
        plt.axis("off")

plt.show()
```



Data Augmentation

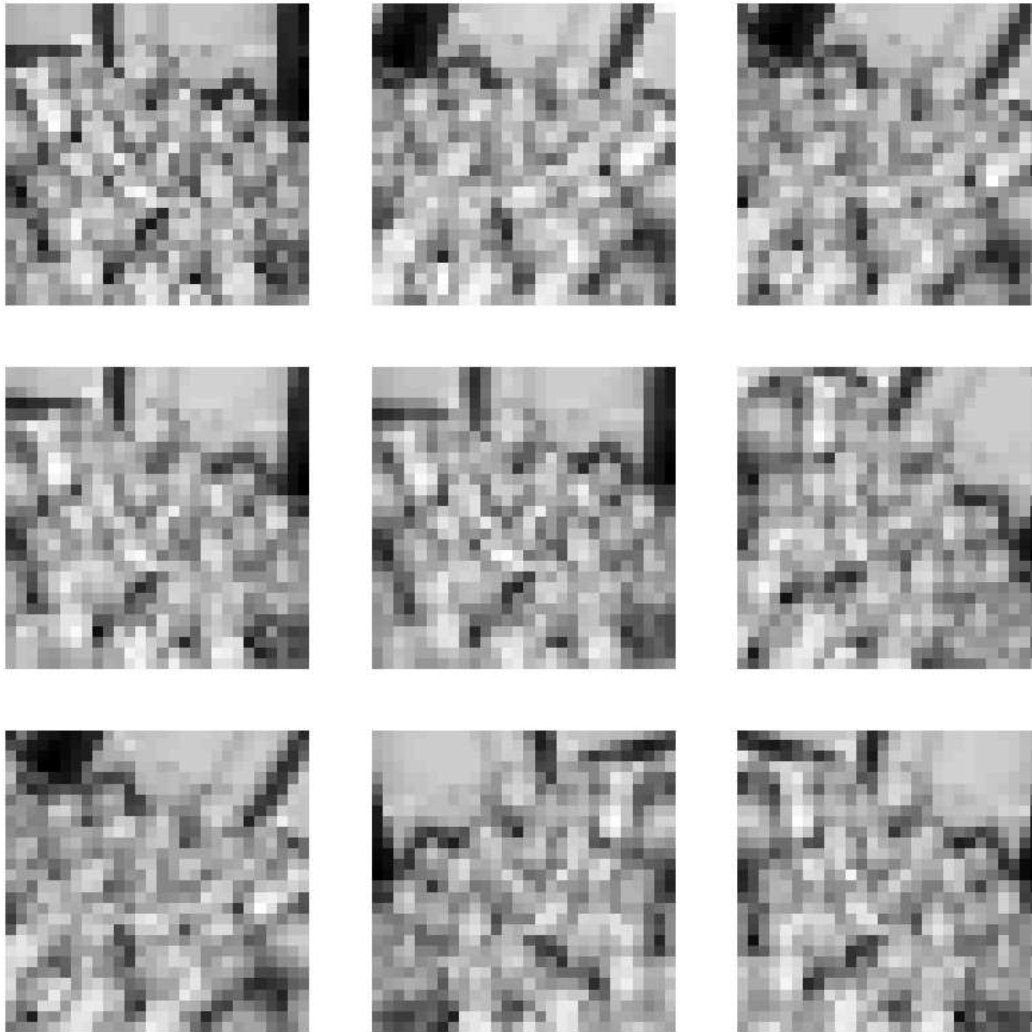
```
data_augmentation_layers = [
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
]

def data_augmentation(images):
    for layer in data_augmentation_layers:
        images = layer(images)
    return images
```

```
plt.figure(figsize=(10, 10))

for images, _ in train_ds.take(1):
    for i in range(9):
        augmented_images = data_augmentation(images)
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(np.array(augmented_images[0]).astype("uint8").squeeze(), cmap="gray")
        plt.axis("off")

plt.show()
```



Improved CNN Model with BN and Dropout

```

num_classes = len(class_names)

model = Sequential([
    layers.Lambda(data_augmentation, input_shape=(28, 28, 1)),
    layers.Rescaling(1./255),

    Conv2D(32, (3, 3), padding='same', activation=None),
    BatchNormalization(),
    Activation('relu'),
    MaxPooling2D((2, 2)),
    Dropout(0.25),

    Conv2D(64, (3, 3), padding='same', activation=None),
    BatchNormalization(),
    Activation('relu'),
    MaxPooling2D((2, 2)),
    Dropout(0.25),

    Conv2D(128, (3, 3), padding='same', activation=None),
    BatchNormalization(),
    Activation('relu'),
    MaxPooling2D((2, 2)),
    Dropout(0.25),

    Conv2D(256, (3, 3), padding='same', activation=None),
    BatchNormalization(),
    Activation('relu'),
    MaxPooling2D((2, 2)),
    Dropout(0.25),

    Flatten(),

```

```
Dense(512, activation=None),
BatchNormalization(),
Activation('relu'),
Dropout(0.5),

Dense(256, activation=None),
BatchNormalization(),
Activation('relu'),
Dropout(0.5),

Dense(128, activation=None),
BatchNormalization(),
Activation('relu'),
Dropout(0.5),

Dense(64, activation=None),
BatchNormalization(),
Activation('relu'),
Dropout(0.5),

Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

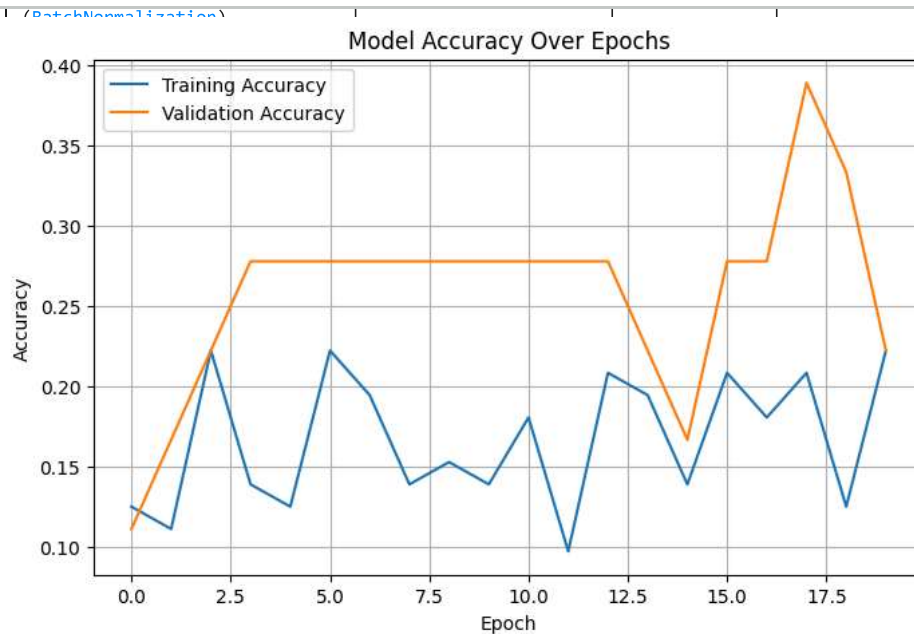


```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20
)
```

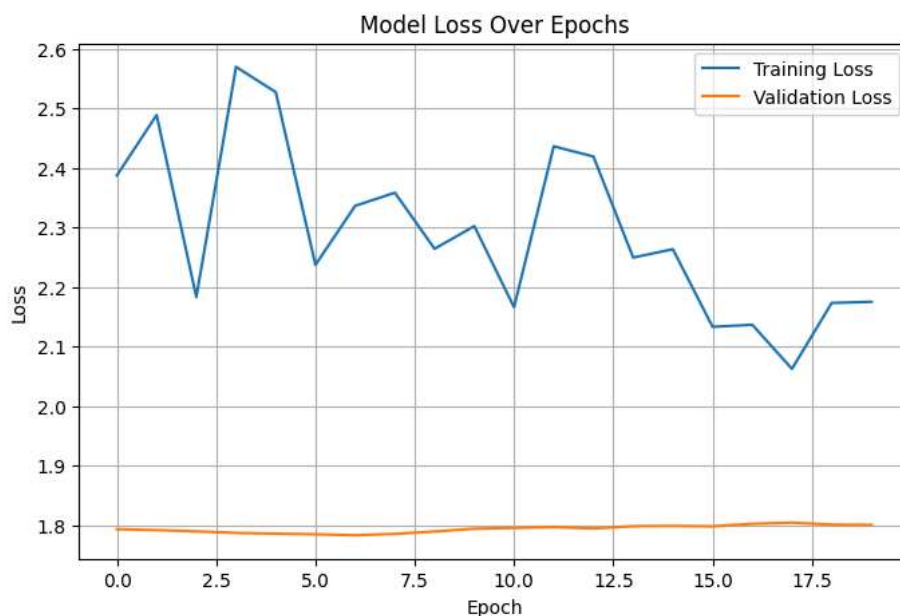
batch_normalization_4 (BatchNormalization)	(None, 20, 20, 32)	120
Epoch 11/20	9s 304ms/step - accuracy: 0.1250 - loss: 2.3874 - val_accuracy: 0.1111 - val_loss: 1.7932	
Epoch 12/20	activation_4 (Activation)	(None, 28, 28, 32) 0
Epoch 13/20	0s 87ms/step - accuracy: 0.1111 - loss: 2.4889 - val_accuracy: 0.1667 - val_loss: 1.7917	
Epoch 14/20	max_pooling2d_4 (MaxPooling2D)	(None, 14, 14, 32) 0
Epoch 15/20	0s 87ms/step - accuracy: 0.2222 - loss: 2.1831 - val_accuracy: 0.2222 - val_loss: 1.7896	
Epoch 16/20	dropout_4 (Dropout)	(None, 14, 14, 32) 0
Epoch 17/20	0s 94ms/step - accuracy: 0.1389 - loss: 2.5698 - val_accuracy: 0.2778 - val_loss: 1.7870	
Epoch 18/20	conv2d_4 (Conv2D)	(None, 14, 14, 64) 18,496
Epoch 19/20	1s 86ms/step - accuracy: 0.1250 - loss: 2.5273 - val_accuracy: 0.2778 - val_loss: 1.7858	
Epoch 20/20	batch_normalization_5 (BatchNormalization)	(None, 14, 14, 64) 256
Epoch 1/20	0s 98ms/step - accuracy: 0.2222 - loss: 2.2373 - val_accuracy: 0.2778 - val_loss: 1.7847	
Epoch 2/20	activation_5 (Activation)	(None, 14, 14, 64) 0
Epoch 3/20	0s 87ms/step - accuracy: 0.1944 - loss: 2.3364 - val_accuracy: 0.2778 - val_loss: 1.7830	
Epoch 4/20	max_pooling2d_5 (MaxPooling2D)	(None, 7, 7, 64) 0
Epoch 5/20	0s 106ms/step - accuracy: 0.1389 - loss: 2.3584 - val_accuracy: 0.2778 - val_loss: 1.7854	
Epoch 6/20	dropout_5 (Dropout)	(None, 7, 7, 64) 0
Epoch 7/20	1s 160ms/step - accuracy: 0.1528 - loss: 2.2642 - val_accuracy: 0.2778 - val_loss: 1.7893	
Epoch 8/20	conv2d_5 (Conv2D)	(None, 7, 7, 128) 73,856
Epoch 9/20	0s 145ms/step - accuracy: 0.1389 - loss: 2.3023 - val_accuracy: 0.2778 - val_loss: 1.7940	
Epoch 10/20	batch_normalization_6 (BatchNormalization)	(None, 7, 7, 128) 512
Epoch 11/20	1s 163ms/step - accuracy: 0.1806 - loss: 2.1662 - val_accuracy: 0.2778 - val_loss: 1.7954	
Epoch 12/20	activation_6 (Activation)	(None, 9, 9, 128) 0
Epoch 13/20	0s 141ms/step - accuracy: 0.0972 - loss: 2.4363 - val_accuracy: 0.2778 - val_loss: 1.7967	
Epoch 14/20	max_pooling2d_6 (MaxPooling2D)	(None, 3, 3, 128) 0
Epoch 15/20	0s 164ms/step - accuracy: 0.2083 - loss: 2.4191 - val_accuracy: 0.2778 - val_loss: 1.7944	
Epoch 16/20	dropout_6 (Dropout)	(None, 3, 3, 128) 0
Epoch 17/20	0s 86ms/step - accuracy: 0.1944 - loss: 2.2495 - val_accuracy: 0.2222 - val_loss: 1.7982	
Epoch 18/20	conv2d_6 (Conv2D)	(None, 3, 3, 256) 295,168
Epoch 19/20	0s 90ms/step - accuracy: 0.1389 - loss: 2.2634 - val_accuracy: 0.1667 - val_loss: 1.7987	
Epoch 20/20	batch_normalization_7 (BatchNormalization)	(None, 3, 3, 256) 1,024
Epoch 1/20	0s 88ms/step - accuracy: 0.1806 - loss: 2.1366 - val_accuracy: 0.2778 - val_loss: 1.8023	
Epoch 2/20	activation_7 (Activation)	(None, 3, 3, 256) 0
Epoch 3/20	0s 93ms/step - accuracy: 0.2083 - loss: 2.0625 - val_accuracy: 0.3889 - val_loss: 1.8042	
Epoch 4/20	max_pooling2d_7 (MaxPooling2D)	(None, 1, 1, 256) 0
Epoch 5/20	0s 98ms/step - accuracy: 0.1250 - loss: 2.1732 - val_accuracy: 0.3333 - val_loss: 1.8009	
Epoch 6/20	dropout_7 (Dropout)	(None, 1, 1, 256) 0
Epoch 7/20	0s 88ms/step - accuracy: 0.2222 - loss: 2.1752 - val_accuracy: 0.2222 - val_loss: 1.8005	
Epoch 8/20	flatten_1 (Flatten)	(None, 256) 0

```
plt.figure(figsize=(8, 5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()
```

batch_normalization_5 (BatchNormalization)	(None, 256)	1,024
activation_5 (Activation)	(None, 256)	0
dropout_5 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 128)	32,896
batch_normalization_6 (BatchNormalization)	(None, 128)	512
activation_6 (Activation)	(None, 128)	0
dropout_6 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 64)	8,256
batch_normalization_7	(None, 64)	256



```
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```



```
test_loss, test_acc = model.evaluate(test_ds)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")
```

```
1/1 ————— 0s 65ms/step - accuracy: 0.1667 - loss: 1.7962
Test Loss: 1.7962
Test Accuracy: 0.1667
```

```
predictions = model.predict(test_ds)
predicted_labels = np.argmax(predictions, axis=1)

print("Predicted labels:")
print(predicted_labels[:20])
```

```
1/1 ————— 0s 299ms/step
Predicted labels:
[4 4 4 4 4 4 4 4 4 4 5 4 4 5 4 4 4 4 4 4]
```

```
y_true = np.concatenate([y for x, y in test_ds], axis=0)

print(classification_report(y_true, predicted_labels, target_names=class_names))
```

	precision	recall	f1-score	support
acai	0.00	0.00	0.00	5
cupuacu	0.00	0.00	0.00	5
graviola	0.00	0.00	0.00	5
guarana	0.00	0.00	0.00	5
pupunha	0.18	1.00	0.30	5
tucuma	0.00	0.00	0.00	5
accuracy			0.17	30
macro avg	0.03	0.17	0.05	30
weighted avg	0.03	0.17	0.05	30

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for classes in labels [0, 1, 2, 3, 4] with no predicted samples.
_warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for classes in labels [0, 1, 2, 3, 4] with no predicted samples.
_warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for classes in labels [0, 1, 2, 3, 4] with no predicted samples.
_warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
```

```
vgg_image_size = (224, 224)
batch_size = 32

train_vgg = keras.utils.image_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset="training",
    seed=1337,
    image_size=vgg_image_size,
    batch_size=batch_size,
    color_mode="rgb"
)

val_vgg = keras.utils.image_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset="validation",
    seed=1337,
    image_size=vgg_image_size,
    batch_size=batch_size,
    color_mode="rgb"
)

test_vgg = keras.utils.image_dataset_from_directory(
    test_dir,
    image_size=vgg_image_size,
    batch_size=batch_size,
    shuffle=False,
    color_mode="rgb"
)
```

```
Found 90 files belonging to 6 classes.
Using 72 files for training.
Found 90 files belonging to 6 classes.
Using 18 files for validation.
Found 30 files belonging to 6 classes.
```

```
base_model = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16\_weights\_tf\_dim\_ordering\_tf\_kernels/58889256/58889256 ————— 0s 0us/step
```

```
for layer in base_model.layers:
    layer.trainable = False
```

```
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
x = Dense(num_classes, activation='softmax')(x)
```

```
model_vgg = Model(inputs=base_model.input, outputs=x)
```

```
model_vgg.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model_vgg.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense_5 (Dense)	(None, 1024)	525,312
dense_6 (Dense)	(None, 6)	6,150

```
train_vgg = keras.utils.image_dataset_from_directory(
    "/content/dataset/FruitinAmazon/train",
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(224,224),
    batch_size=32,
    color_mode="rgb"
)
```

```
val_vgg = keras.utils.image_dataset_from_directory(
    "/content/dataset/FruitinAmazon/train",
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(224,224),
    batch_size=32,
    color_mode="rgb"
)
```